

Roadmap de Développement — Application SaaS de Gestion Comptable

Version du document : 1.0

Date de publication : 13 septembre 2026

Durée totale estimée : 18-24 mois

Responsable : Équipe Produit & Architecture

Statut général : En cours de planification

Table des matières

1. [Vision et Objectifs Globaux](#)
 2. [Métriques de Progression Récapitulatives](#)
 3. [Jalons Critiques](#)
 4. [Phase 0 — Setup Initial](#)
 5. [Phase 1 — Core MVP](#)
 6. [Phase 2 — Module Facturation](#)
 7. [Phase 3 — Module Trésorerie](#)
 8. [Phase 4 — Module Conformité](#)
 9. [Phase 5 — Module IA](#)
 10. [Phase 6 — Infrastructure & Ops](#)
 11. [Plan de Mitigation des Risques](#)
 12. [Glossaire](#)
-

1. Vision et Objectifs Globaux

L'application SaaS de gestion comptable vise à offrir une solution complète, modulaire et évolutive pour les petites et moyennes entreprises (PME), les cabinets comptables et les indépendants. La plateforme couvrira l'ensemble du cycle comptable : de la saisie des transactions à la production de bilans, en passant par la facturation, la gestion de trésorerie et la conformité fiscale.

Objectifs stratégiques :

- Offrir une expérience utilisateur intuitive réduisant le temps de prise en main à moins de 2 heures
 - Garantir la conformité réglementaire avec les normes comptables françaises et européennes
 - Intégrer l'intelligence artificielle pour automatiser les tâches répétitives (saisie, catégorisation, prédiction)
 - Assurer une scalabilité technique permettant de passer de 100 à 100 000 utilisateurs sans refonte majeure
 - Fournir un système de reporting en temps réel avec des tableaux de bord personnalisables
-

2. Métriques de Progression Récapitulatives

Phase	Nom	Progression Globale	Durée Estimée	Statut
Phase 0	Setup Initial	0%	3-4 semaines	À démarrer
Phase 1	Core MVP	0%	10-14 semaines	À démarrer
Phase 2	Module Facturation	0%	8-10 semaines	Planifiée
Phase 3	Module Trésorerie	0%	6-8 semaines	Planifiée
Phase 4	Module Conformité	0%	8-10 semaines	Planifiée
Phase 5	Module IA	0%	10-12 semaines	Planifiée
Phase 6	Infrastructure & Ops	0%	6-8 semaines	Planifiée
TOTAL	Projet Complet	[] 0%	51-66 semaines	En planification

Progression Cumulée par Module

Mois	Phase Active	Progression Cumulée (Jours)	Livrables Clés
Mois 1	Phase 0	0%	Environnement, CI/CD, Auth
Mois 2-3	Phase 1	15%	MVP Utilisateurs, Transactions, Dashboard
Mois 4-5	Phase 2	32%	Facturation, Devis, Portail Client
Mois 6-7	Phase 3	52%	Cash-flow, Dashboard 360°, Alertes
Mois 8-9	Phase 4	70%	TVA, Bilan, FEC, Audit
Mois 10-12	Phase 5	85%	OCR, ML Catégorisation, Prédictions
Mois 12-14	Phase 6	100%	Docker, Monitoring, Tests Complets

3. Jalons Critiques

Jalon	Date Cible	Phase	Description	Critère de Validation
J0 — Kick-off	15/09/2026	Phase 0	Lancement officiel du projet	Réunion de lancement tenue, équipe constituée
J1 — Environment Ready	06/10/2026	Phase 0	Environnement de développement opérationnel	CI/CD fonctionne, base de données déployée
J2 — Auth Opérationnelle	20/10/2026	Phase 0	Système d'authentification fonctionnel	Inscription, connexion, reset mot de passe testés
J3 — MVP Démo	15/12/2026	Phase 1	Démonstration du MVP aux parties prenantes	Utilisateurs, transactions, dashboard fonctionnels
J4 — Beta Ouverte	15/02/2027	Phase 2	Ouverture en bêta à 50 utilisateurs pilotes	Facturation + Portail Client accessibles

Jalon	Date Cible	Phase	Description	Critère de Validation
J5 — Lancement V1	01/05/2027	Phase 2-3	Lancement production V1	Facturation, Trésorerie, Conformité de base
J6 — Conformité Certifiée	01/08/2027	Phase 4	Validation conformité TVA/FEC	Audit externe réussi
J7 — IA Beta	15/10/2027	Phase 5	Démonstration fonctionnalités IA	OCR et catégorisation ML opérationnels
J8 — Production Scaling	15/01/2028	Phase 6	Déploiement production à grande échelle	Monitoring, Docker, CI/CD en production
J9 — GA (General Availability)	01/04/2028	Phase 6	Lancement commercial complet	Tous les modules validés, SLA défini

4. Phase 0 — Setup Initial

Progression Globale : [] 0%

Durée estimée : 3-4 semaines

Objectif : Établir l'infrastructure technique fondamentale, l'arborescence du projet, les outils de développement et le système d'authentification de base.

4.1 Arborescence et Configuration du Projet

[] Tâche 0.1 — Initialisation du dépôt Git et structure de projet

- **Description complète :** Créer le dépôt Git initial avec une structure de répertoires claire et maintenable. Définir les conventions de nommage, les branches strategies (Git Flow), les fichiers `.gitignore`, `.editorconfig`, et les templates de commits (Conventional Commits). La structure doit séparer clairement le frontend, le backend, les configurations et la documentation.
- **Dépendances :** Aucune (tâche initiale)
- **Critères d'acceptation :**
 - ☐ Dépôt Git créé avec README.md complet
 - ☐ Structure de répertoires validée par l'équipe
 - ☐ `.gitignore` couvre Node.js, Python, IDE, OS
 - ☐ Conventional Commits documentés et adoptés
 - ☐ Branche `main` protégée, branches `develop`, `feature/*`, `hotfix/*` configurées
- **Estimation :** 2 jours/hommes
- **Risques identifiés :**
 - **Risque faible :** Désaccord sur la structure de répertoires → Mitigation : Atelier de 2h en début de phase pour valider l'architecture
 - **Risque faible :** Oubli de fichiers sensibles dans le dépôt → Mitigation : Review exhaustive du `.gitignore` + `git-secrets`

- **Alternatives techniques :**

- **Alternative A :** Monorepo avec packages séparés (Nx, Turborepo) → Plus complexe mais évolutif
 - **Alternative B :** Polyrepo avec repo séparé par service → Simpler initialement mais coordination accrue
-

[] Tâche 0.2 — Configuration de l'environnement de développement local

- **Description complète :** Mettre en place un environnement de développement unifié pour tous les développeurs. Cela inclut l'installation et la configuration des outils essentiels (Node.js, Python, Docker, Docker Compose), les fichiers `.env.example`, les scripts de démarrage rapide (`make dev` ou `npm run dev`), et la documentation d'installation. L'objectif est que tout nouveau développeur puisse démarrer en moins de 30 minutes.
 - **Dépendances :** Tâche 0.1
 - **Critères d'acceptation :**
 - ☐ Script de démarrage fonctionnel sur machines Windows et Mac
 - ☐ Docker Compose configuré avec tous les services de base
 - ☐ Fichier `.env.example` complet avec toutes les variables nécessaires
 - ☐ Documentation d'installation dans `docs/setup.md`
 - ☐ Au moins 2 développeurs ont validé le setup sur leur machine
 - **Estimation :** 3 jours/hommes
 - **Risques identifiés :**
 - **Risque moyen :** Incompatibilités OS entre membres de l'équipe → Mitigation : Test sur Windows, Mac et Linux
 - **Risque moyen :** Docker non disponible ou version obsolète → Mitigation : Documentation des versions requises + fallback sans Docker
 - **Alternatives techniques :**
 - **Alternative A :** Dev containers VS Code → Environnement reproductible mais nécessite VS Code
 - **Alternative B :** Scripts d'installation manuels par OS → Plus flexible mais moins reproductible
-

[] Tâche 0.3 — Configuration CI/CD (Intégration Continue / Déploiement Continu)

- **Description complète :** Configurer un pipeline CI/CD complet avec GitHub Actions (ou GitLab CI). Le pipeline doit inclure : linting automatique, tests unitaires, build, analysis de code (SonarQube/SonarCloud), et déploiement automatique sur un environnement de staging. La configuration doit supporter les PR checks et le déploiement sur `main` vers la production.
- **Dépendances :** Tâches 0.1, 0.2
- **Critères d'acceptation :**
 - ☐ Pipeline CI déclenché automatiquement sur PR
 - ☐ Linting, tests unitaires, et build passent dans le pipeline

- ☐ Déploiement automatique sur staging après merge sur **develop**
 - ☐ Badges de statut visibles dans le README
 - ☐ Pipeline exécuté avec succès au moins 3 fois
 - **Estimation** : 4 jours/hommes
 - **Risques identifiés** :
 - **Risque moyen** : Pipeline trop lent (> 15 min) → Mitigation : Parallélisation des jobs, caching des dépendances
 - **Risque moyen** : Secrets exposés dans les logs → Mitigation : Utilisation des secrets GitHub Actions, mask des variables sensibles
 - **Alternatives techniques** :
 - **Alternative A** : GitLab CI → Intégré à GitLab mais moins flexible pour du multi-OS
 - **Alternative B** : Jenkins → Plus puissant mais maintenance lourde
 - **Alternative C** : Vercel/Netlify pour le frontend + GitHub Actions pour le backend → Spécialisé mais couplage limité
-

[] Tâche 0.4 — Configuration de la base de données et ORM

- **Description complète** : Configurer la base de données PostgreSQL (production) et SQLite/PostgreSQL (développement). Mettre en place l'ORM (Prisma, TypeORM, ou SQLAlchemy selon la stack) avec les premiers modèles de données (User, Organization, Transaction). Configurer les migrations automatiques, les seeds pour le développement, et les scripts de sauvegarde.
- **Dépendances** : Tâches 0.1, 0.2
- **Critères d'acceptation** :
 - ☐ Base PostgreSQL fonctionnelle en local via Docker
 - ☐ ORM configuré avec connexion réussie
 - ☐ Au moins 5 modèles de base créés (User, Org, Transaction, Category, Account)
 - ☐ Migrations automatiques fonctionnelles
 - ☐ Script de seed avec données de test opérationnel
 - ☐ Tests de migration aller/retour validés
- **Estimation** : 3 jours/hommes
- **Risques identifiés** :
 - **Risque moyen** : Incompatibilité d'ORM avec certaines fonctionnalités SQL avancées → Mitigation : Validation préalable des requêtes complexes
 - **Risque faible** : Performances des migrations en production → Mitigation : Migrations horizontales, index préalables
- **Alternatives techniques** :
 - **Alternative A** : Prisma (TypeScript) → Excellent DX mais overhead de compilation
 - **Alternative B** : TypeORM (TypeScript) → Plus flexible mais maintenance plus lourde

- **Alternative C** : SQLAlchemy (Python) si backend Python → Mature mais stack différente
 - **Alternative D** : MongoDB → Schema-less mais inadapté aux données comptables structurées
-

[] Tâche 0.5 — Authentification de base (Inscription, Connexion, Reset)

- **Description complète** : Implémenter le système d'authentification complet : inscription par email/mot de passe avec validation email, connexion, reset de mot de passe par email, déconnexion, et gestion des sessions. Intégrer JWT (JSON Web Tokens) pour l'API et cookies sécurisés pour le frontend. Implémenter la gestion des rôles utilisateur (Admin, Comptable, Lecture seule) et la logique multi-organisations (un utilisateur peut appartenir à plusieurs organisations).
 - **Dépendances** : Tâches 0.1, 0.2, 0.4
 - **Critères d'acceptation** :
 - ☐ Inscription avec email + mot de passe fonctionnelle
 - ☐ Vérification email obligatoire avant accès
 - ☐ Connexion avec JWT + refresh token
 - ☐ Reset mot de passe par email fonctionnel
 - ☐ Déconnexion invalide les tokens côté serveur
 - ☐ Rôles (Admin, Comptable, Lecture seule) opérationnels
 - ☐ Multi-tenancy (organisation) fonctionnelle
 - ☐ Tests de sécurité : SQL injection, XSS, CSFR validés
 - ☐ Rate limiting sur les endpoints d'authentification
 - **Estimation** : 5 jours/hommes
 - **Risques identifiés** :
 - **Risque élevé** : Vulnérabilité dans le système d'authentification → Mitigation : Audit de sécurité, utilisation de bibliothèques éprouvées (Passport.js, Auth0 comme fallback), tests de pénétration
 - **Risque moyen** : Gestion des tokens JWT complexe (refresh, revocation) → Mitigation : Blacklist de tokens Redis, rotation des refresh tokens
 - **Risque moyen** : Envoi d'emails en production (SMTP) → Mitigation : Service email tiers (SendGrid, Mailgun), sandbox en développement
 - **Alternatives techniques** :
 - **Alternative A** : Auth0 / Clerk / Supabase Auth → Externalisé, rapide mais coût récurrent et dépendance tierce
 - **Alternative B** : OAuth2 / SSO uniquement → Suffisant pour B2B mais exclut les particuliers
 - **Alternative C** : Session-based auth → Plus simple mais moins scalable pour SPA/API
-

[] Tâche 0.6 — Architecture de base et configuration des services fondamentaux

- **Description complète** : Définir et implémenter l'architecture de base de l'application : structure du backend (API REST ou GraphQL), configuration du serveur web (Nginx reverse proxy), configuration CORS, logging structuré (Winston/Pino), gestion des erreurs centralisée, validation des entrées

(Zod/Joi/Yup), et configuration de la file de messages (RabbitMQ ou Redis Pub/Sub pour les tâches asynchrones futures).

- **Dépendances** : Tâches 0.1, 0.2, 0.5
 - **Critères d'acceptation** :
 - ☐ API REST (ou GraphQL) avec endpoints de test fonctionnels
 - ☐ CORS correctement configuré pour les origines dev et prod
 - ☐ Logging structuré avec niveaux (debug, info, warn, error)
 - ☐ Gestion des erreurs centralisée avec codes HTTP cohérents
 - ☐ Validation des entrées sur tous les endpoints
 - ☐ Redis configuré pour cache et queue de messages
 - ☐ Documentation API (Swagger/OpenAPI) générée automatiquement
 - **Estimation** : 4 jours/hommes
 - **Risques identifiés** :
 - **Risque moyen** : Choix d'architecture trop rigide → Mitigation : Architecture hexagonale/clean code pour permettre l'évolution
 - **Risque faible** : Configuration CORS incorrecte en production → Mitigation : Whitelist d'origines strictes, variable d'environnement
 - **Alternatives techniques** :
 - **Alternative A** : REST API → Standard, simple, cacheable
 - **Alternative B** : GraphQL → Flexible pour le frontend mais plus complexe à optimiser
 - **Alternative C** : tRPC → Type-safe mais limité aux stacks TypeScript
-

5. Phase 1 — Core MVP

Progression Globale : [] 0%

Durée estimée : 10-14 semaines

Objectif : Livrer un MVP fonctionnel permettant aux utilisateurs de créer un compte, d'effectuer des transactions de base, et de visualiser un dashboard minimal avec des indicateurs clés de performance comptable.

5.1 Module Utilisateurs et Organisations

[] Tâche 1.1 — Gestion complète des profils utilisateurs

- **Description complète** : Développer l'ensemble des fonctionnalités de gestion de profil : vue profil détaillée avec avatar, nom, prénom, email, téléphone, entreprise, bio ; édition de profil avec formulaire validé ; upload d'avatar (image) avec redimensionnement et stockage S3/Cloudinary ; préférences utilisateur (langue, fuseau horaire, notifications) ; historique des modifications de profil ; et suppression de compte avec confirmation et anonymisation des données.
- **Dépendances** : Tâches 0.4, 0.5, 0.6

- **Critères d'acceptation :**

- ☐ Page profil accessible avec toutes les informations affichées
- ☐ Édition profil avec validation côté client et serveur
- ☐ Upload d'avatar fonctionnel (jpg, png, max 5Mo) avec crop
- ☐ Préférences sauvegardées et appliquées immédiatement
- ☐ Suppression de compte avec confirmation email + délai de grâce 30 jours
- ☐ Tests unitaires et d'intégration couvrant 80% du code

- **Estimation :** 5 jours/hommes

- **Risques identifiés :**

- **Risque moyen :** RGPD — suppression de données incomplète → Mitigation : Audit de toutes les tables, suppression en cascade ou anonymisation pseudonyme
- **Risque faible :** Performance de l'upload d'images → Mitigation : CDN, lazy loading, images optimisées

- **Alternatives techniques :**

- **Alternative A :** Cloudinary pour les images → Redimensionnement automatique, mais dépendance externe
- **Alternative B :** S3 + Lambda pour le traitement → Plus contrôlé mais plus complexe
- **Alternative C :** Stockage local avec nginx → Simple mais non scalable

[] Tâche 1.2 — Gestion des organisations et équipes

- **Description complète :** Implémenter le système multi-organisations permettant à chaque utilisateur de créer, rejoindre, et gérer des organisations. Fonctionnalités : création d'organisation avec nom, SIRET, secteur d'activité ; invitation de membres par email avec rôle assigné (Admin, Comptable, Lecture seule) ; gestion des invitations en attente ; modification/suppression de membres ; transfer de propriété ; et leave/exit organization. Chaque organisation est isolée dans la base de données (multi-tenancy par row-level security).

- **Dépendances :** Tâches 0.4, 0.5, 0.6, 1.1

- **Critères d'acceptation :**

- ☐ Création d'organisation avec toutes les informations requises
- ☐ Invitation par email avec lien valide 7 jours
- ☐ 3 rôles distincts avec permissions différentes
- ☐ Row-level security actif : chaque utilisateur ne voit que ses organisations
- ☐ Transfer de propriété avec confirmation et délai de sécurité
- ☐ Un utilisateur peut appartenir à maximum 10 organisations
- ☐ Tests d'isolement multi-tenancy validés

- **Estimation :** 6 jours/hommes

- **Risques identifiés :**

- **Risque élevé** : Fuite de données entre organisations → Mitigation : Tests d'intégration systématiques de séparation, RLS PostgreSQL, audit régulier
- **Risque moyen** : Gestion des invitations expirées → Mitigation : Cron de nettoyage, notifications avant expiration

- **Alternatives techniques :**

- **Alternative A** : Row-Level Security PostgreSQL → Isolation forte au niveau DB
- **Alternative B** : Schema séparé par organisation → Isolation totale mais maintenance lourde
- **Alternative C** : Base séparée par organisation → Maximum d'isolation mais complexité opérationnelle

[] Tâche 1.3 — Système de Plan Comptable et Catégories

- **Description complète** : Créer un système de plan comptable configurable permettant aux organisations de définir leurs catégories comptables. Inclure : plan comptable français standard (PCG) pré-chargé avec les 10 classes ; possibilité de créer des sous-catégories personnalisées ; assignation de catégories aux transactions ; tags et labels personnalisables ; et archive/mise à jour des plans comptables historiques avec versioning.
- **Dépendances** : Tâches 0.4, 0.6, 1.2
- **Critères d'acceptation** :
 - ☐ Plan comptable PCG pré-chargé avec les 10 classes et 412 comptes standards
 - ☐ Possibilité de créer des catégories personnalisées illimitées
 - ☐ Arborescence illimitée de sous-catégories (depth max 5)
 - ☐ Tags et labels assignables aux transactions
 - ☐ Versioning du plan comptable (historique des modifications)
 - ☐ Import/Export du plan comptable en CSV
- **Estimation** : 5 jours/hommes
- **Risques identifiés** :
 - **Risque moyen** : Complexité du plan comptable PCG → Mitigation : Mapping simplifié pour les PME, plan comptable international comme alternative
 - **Risque faible** : Confusion des catégories → Mitigation : Interface intuitive avec preview, catégories favorites
- **Alternatives techniques** :
 - **Alternative A** : Plan comptable fixe → Simple mais non flexible
 - **Alternative B** : Catégories basées sur l'IA → Automatique mais nécessite Phase 5

5.2 Module Transactions

[] Tâche 1.4 — Saisie et gestion des transactions manuelles

- **Description complète** : Développer l'interface de saisie manuelle des transactions pour les utilisateurs. Inclure : formulaire de création de transaction (date, montant, description, catégorie, compte, type : débit/crédit, pièce justificative) ; liste des transactions avec recherche, filtres (date, catégorie, montant, organisation) et tri ; édition et suppression de transactions ; double écriture comptable (débit et crédit liés) ; et import de transactions par CSV/OFX/QIF.
 - **Dépendances** : Tâches 0.4, 0.6, 1.1, 1.2, 1.3
 - **Critères d'acceptation** :
 - ☐ Formulaire de transaction complet et validé
 - ☐ Liste des transactions avec recherche textuelle et 5 filtres
 - ☐ Modification et suppression avec confirmation
 - ☐ Double écriture comptable vérifiée (total débits = total crédits)
 - ☐ Import CSV/OFX/QIF avec mapping des colonnes
 - ☐ Export des transactions en CSV et PDF
 - **Estimation** : 8 jours/hommes
 - **Risques identifiés** :
 - **Risque élevé** : Erreurs comptables dans la saisie manuelle → Mitigation : Validation en temps réel, double écriture, alertes de déséquilibre
 - **Risque moyen** : Import de fichiers corrompus → Mitigation : Validation du format, preview avant import, rollback en cas d'erreur
 - **Alternatives techniques** :
 - **Alternative A** : Saisie simple (débit/crédit) → Plus simple mais moins rigoureux
 - **Alternative B** : Import bancaire uniquement (OCR/API) → Automatisé mais nécessite Phase 5
-

[] Tâche 1.5 — Connexion bancaire (Open Banking / API bancaire)

- **Description complète** : Implémenter la connexion aux comptes bancaires via les API d'Open Banking (PSD2 en Europe). Inclure : authentification auprès des banques via les agrégateurs (Plaid, TrueLayer, ou Yodlee) ; récupération automatique des transactions ; mapping automatique des transactions bancaires vers les catégories comptables ; gestion des réconciliations ; et support de multiples banques.
- **Dépendances** : Tâches 0.5, 0.6, 1.2, 1.3, 1.4
- **Critères d'acceptation** :
 - ☐ Connexion à au moins 3 banques via agrégateur fonctionnelle
 - ☐ Récupération automatique des transactions (dernier 90 jours)
 - ☐ Mapping automatique avec taux de réussite > 70%
 - ☐ Réconciliation manuelle et automatique
 - ☐ Gestion des sessions d'authentification bancaire (token refresh)
 - ☐ Support multi-banques (minimum 3)
- **Estimation** : 10 jours/hommes

- **Risques identifiés :**

- **Risque élevé :** Dépendance à des API tierces (Plaid, etc.) → Mitigation : Abstraction layer, fallback manuel, multi-fournisseur
- **Risque élevé :** Disponibilité des API bancaires → Mitigation : Cache local, retry avec backoff exponentiel, mode dégradé
- **Risque moyen :** Sécurité des données bancaires → Mitigation : Chiffrement au repos, tokens sécurisés, audit de sécurité

- **Alternatives techniques :**

- **Alternative A :** Plaid → Couverture US forte, limité en Europe
 - **Alternative B :** TrueLayer → Excellent en Europe, PSD2 natif
 - **Alternative C :** Développement maison (screen scraping) → Dangereux, non conforme PSD2, déconseillé
 - **Alternative D :** Yodlee → Global mais coûteux
-

5.3 Dashboard Minimal

[] Tâche 1.6 — Dashboard minimal avec KPIs comptables

- **Description complète :** Créer un tableau de bord utilisateur affichant les indicateurs clés de performance comptable. Inclure : solde total des comptes ; revenus et dépenses du mois en cours (et comparaison avec mois précédent) ; graphique d'évolution mensuelle des dépenses (12 derniers mois) ; top 5 des catégories de dépenses ; transactions récentes ; et alertes de besoin d'attention (découvert, facture échéante, etc.). Le dashboard doit être responsive et accessible (WCAG 2.1 AA).
- **Dépendances :** Tâches 0.4, 0.6, 1.1, 1.2, 1.3, 1.4
- **Critères d'acceptation :**
 - ☐ Solde total calculé et affiché en temps réel
 - ☐ Revenus/Dépenses du mois avec comparaison M-1
 - ☐ Graphique en barres (12 mois) avec Chart.js ou équivalent
 - ☐ Top 5 catégories de dépenses
 - ☐ Liste des 10 dernières transactions
 - ☐ Design responsive (mobile, tablette, desktop)
 - ☐ Accessibilité WCAG 2.1 AA validée (contraste, navigation clavier, screen reader)
 - ☐ Temps de chargement < 2 secondes
- **Estimation :** 7 jours/hommes
- **Risques identifiés :**
 - **Risque moyen :** Performance des graphiques avec gros volumes → Mitigation : Agrégation serveur, pagination, virtualisation
 - **Risque faible :** Inaccessibilité des graphiques → Mitigation : Texte alternatif, tableaux de données accessibles
- **Alternatives techniques :**

- **Alternative A** : Chart.js → Léger, flexible, communautaire
 - **Alternative B** : Recharts → Plus riche mais plus lourd
 - **Alternative C** : D3.js → Maximum de contrôle mais complexité élevée
 - **Alternative D** : Composants SaaS (MUI Charts, Ant Design) → Intégré mais dépendance forte
-

[] Tâche 1.7 — Gestion des comptes et soldes

- **Description complète** : Implémenter le module de gestion des comptes bancaires et comptables : ajout de comptes (banque, cash, carte bancaire, tiers) ; saisie des soldes initiaux ; suivi des soldes en temps réel ; rapprochement bancaire (matching des transactions importées avec les transactions saisies manuellement) ; et état des comptes avec détail des opérations.
 - **Dépendances** : Tâches 0.4, 0.6, 1.3, 1.4, 1.5, 1.6
 - **Critères d'acceptation** :
 - ☐ Ajout de comptes avec type, solde initial, et devise
 - ☐ Suivi des soldes mis à jour automatiquement
 - ☐ Rapprochement bancaire avec matching automatique et manuel
 - ☐ État des comptes avec détail filtrable
 - ☐ Support multi-devises avec taux de change
 - **Estimation** : 6 jours/hommes
 - **Risques identifiés** :
 - **Risque moyen** : Rapprochement bancaire complexe (doublons, décalages) → Mitigation : Algorithme de matching en 3 étapes (exact, flou, manuel)
 - **Risque faible** : Conversion de devises inexacte → Mitigation : API de taux de change (exchangerate.host), cache 1h
 - **Alternatives techniques** :
 - **Alternative A** : Rapprochement automatique total → Ambitieux mais risqué
 - **Alternative B** : Rapprochement semi-automatique (suggère, l'utilisateur confirme) → Plus sûr et user-friendly
 - **Alternative C** : Aucun rapprochement → Simple mais non professionnel
-

[] Tâche 1.8 — Système de notifications et alertes basique

- **Description complète** : Implémenter un système de notifications internes et par email pour alertes critiques : solde insuffisant, facture échéante proche, transaction inhabituelle de grande valeur, et rappel d'échéances fiscales. Le système doit permettre à l'utilisateur de configurer ses préférences de notification (email, in-app, push) et de gérer un historique des notifications reçues.
- **Dépendances** : Tâches 0.5, 0.6, 1.1, 1.6
- **Critères d'acceptation** :
 - ☐ Notifications in-app avec badge count

- ☐ Notifications email pour alertes critiques
 - ☐ 4 types d'alertes configurés (solde, facture, transaction, fiscal)
 - ☐ Centre de préférences de notification
 - ☐ Historique des notifications avec marquage lu/non lu
 - ☐ Tests de délai et de justesse des alertes validés
- **Estimation** : 4 jours/hommes
 - **Risques identifiés** :
 - **Risque moyen** : Trop de notifications (spam) → Mitigation : Configurabilité fine, cooldown, filtrage intelligent
 - **Risque faible** : Emails non délivrés (spam) → Mitigation : Domaine authentifié (SPF, DKIM, DMARC), monitoring de délivrabilité
 - **Alternatives techniques** :
 - **Alternative A** : Firebase Cloud Messaging → Push mobile mais dépendance Google
 - **Alternative B** : Pusher / Ably → Temps réel mais coût récurrent
 - **Alternative C** : WebSocket maison → Contrôle total mais maintenance lourde

Métriques de Progression — Phase 1

Tâche	Statut	Progression
Tâche 1.1 — Gestion des profils	☐	0%
Tâche 1.2 — Organisations et équipes	☐	0%
Tâche 1.3 — Plan comptable et catégories	☐	0%
Tâche 1.4 — Transactions manuelles	☐	0%
Tâche 1.5 — Connexion bancaire	☐	0%
Tâche 1.6 — Dashboard minimal	☐	0%
Tâche 1.7 — Gestion des comptes	☐	0%
Tâche 1.8 — Notifications et alertes	☐	0%
Total Phase 1	[]	0%

6. Phase 2 — Module Facturation

Progression Globale : [] 0%

Durée estimée : 8-10 semaines

Objectif : Permettre la création, gestion et envoi de devis et factures, l'accès client via un portail dédié, et le suivi des paiements.

6.1 Module Devis

[] Tâche 2.1 — Création et gestion des devis

- **Description complète** : Développer un module complet de gestion de devis (quotes) permettant aux utilisateurs de créer des devis professionnels. Fonctionnalités : template de devis personnalisable avec logo et couleurs de l'entreprise ; ligne par ligne avec description, quantité, prix unitaire, TVA, montant total ; calcul automatique des totaux HT, TVA, TTC ; statut du devis (brouillon, envoyé, accepté, refusé, expiré) ; versioning des devis ; import d'éléments depuis le catalogue produits/services ; et génération PDF.
 - **Dépendances** : Tâches 0.4, 0.6, 1.3
 - **Critères d'acceptation** :
 - ☐ Formulaire de création de devis avec lignes dynamiques
 - ☐ Calcul automatique HT/TVA/TTC avec taux TVA configurable
 - ☐ Templates personnalisables (logo, couleurs, mentions légales)
 - ☐ 5 statuts de devis gérés (brouillon, envoyé, accepté, refusé, expiré)
 - ☐ Versioning avec historique complet
 - ☐ Génération PDF de qualité professionnelle
 - ☐ Import depuis catalogue produits/services
 - ☐ Envoi par email intégré avec suivi d'ouverture
 - **Estimation** : 6 jours/hommes
 - **Risques identifiés** :
 - **Risque moyen** : Calcul TVA incorrect selon la juridiction → Mitigation : Base de données des taux TVA par pays/région, validation par expert-comptable
 - **Risque faible** : Templates HTML complexes mal rendus en PDF → Mitigation : Test sur plusieurs générateurs PDF, fallback simple
 - **Alternatives techniques** :
 - **Alternative A** : Puppeteer/Playwright pour PDF → Bon rendu HTML/CSS mais lourd
 - **Alternative B** : pdfkit (Node.js) → Léger mais contrôle limité du rendu
 - **Alternative C** : LibreOffice en ligne de commande → Gratuit mais dépendant de l'installation
 - **Alternative D** : Service API (PDF.co, Boldsign) → Externalisé mais coût par document
-

[] Tâche 2.2 — Catalogue produits et services

- **Description complète** : Créer un catalogue de produits et services réutilisables dans les devis et factures. Inclure : ajout de produits (nom, référence, catégorie, prix HT, TVA applicable, unité, description) ; gestion du stock (quantité disponible, seuil d'alerte) ; catégories de produits ; prix multiples (HT/TTC) ; tarifs spéciaux par client ; historique des prix ; et import/export CSV du catalogue.
- **Dépendances** : Tâches 0.4, 0.6, 1.3, 2.1
- **Critères d'acceptation** :
 - ☐ Ajout de produits avec toutes les informations
 - ☐ Catégorisation des produits

- ☐ Gestion du stock avec seuils d'alerte
 - ☐ Prix HT et TTC associés
 - ☐ Tarifs spéciaux par client
 - ☐ Historique des modifications de prix
 - ☐ Import/Export CSV
 - **Estimation** : 5 jours/hommes
 - **Risques identifiés** :
 - **Risque moyen** : Gestion des prix multiples complexe → Mitigation : Règle de priorité claire, validation manuelle
 - **Risque faible** : Stock non synchronisé → Mitigation : Mise à jour temps réel, notifications de seuil
 - **Alternatives techniques** :
 - **Alternative A** : Catalogue fixe → Simple mais non flexible
 - **Alternative B** : Catalogue intelligent avec suggestions IA → Nécessite Phase 5
-

6.2 Module Factures

[] Tâche 2.3 — Création et gestion des factures

- **Description complète** : Développer le module de facturation complet : création de factures à partir de devis acceptés ou de saisie manuelle ; lignes de facture avec les mêmes fonctionnalités que les devis ; numérotation automatique des factures selon la norme NF525 (chrono, séquentiel, unique) ; gestion des statuts (brouillon, émise, payée, partiellement payée, annulée) ; génération PDF ; et envoi par email avec accusé de réception.
- **Dépendances** : Tâches 0.4, 0.6, 1.3, 2.1, 2.2
- **Critères d'acceptation** :
 - ☐ Création de factures manuelle et depuis devis
 - ☐ Numérotation automatique conforme NF525
 - ☐ Calcul automatique HT/TVA/TTC
 - ☐ 5 statuts de facture gérés
 - ☐ Génération PDF avec watermark « paid » quand payée
 - ☐ Envoi email avec accusé de réception
 - ☐ Annulation de facture avec motif et versioning
 - ☐ Duplique de facture (copie)
- **Estimation** : 7 jours/hommes
- **Risques identifiés** :
 - **Risque élevé** : Non-conformité NF525 → Mitigation : Validation par expert-comptable, audit régulier de la numérotation
 - **Risque moyen** : Annulation/avoir complexe → Mitigation : Flux de validation en 2 étapes, historique complet

- **Alternatives techniques :**

- **Alternative A :** Numérotation par base de données → Simple mais risquée en cas de concurrence
- **Alternative B :** Numérotation par séquence atomique → Plus sûr mais nécessite gestion de transaction

[] Tâche 2.4 — Gestion des avoirs (Avoirs fiscaux et commerciaux)

- **Description complète :** Implémenter le système d'avoirs pour corriger ou annuler des factures émises. Inclure : création d'avoir liée à une facture existante ; calcul automatique du montant restant de la facture ; choix entre avoir commercial (suspension) et avoir fiscal (annulation avec crédit d'impôt) ; workflow de validation ; et intégration avec le module de conformité pour la TVA.
- **Dépendances :** Tâches 0.4, 0.6, 1.3, 2.3
- **Critères d'acceptation :**
 - ☐ Création d'avoir liée à une facture avec montant partiel ou total
 - ☐ Calcul du montant restant à avoir
 - ☐ Distinction avoir commercial vs fiscal
 - ☐ Workflow de validation en 2 étapes
 - ☐ Intégration TVA automatique
 - ☐ Historique des avoirs avec motif obligatoire
- **Estimation :** 4 jours/hommes
- **Risques identifiés :**
 - **Risque élevé :** Erreur de TVA sur avoir → Mitigation : Calcul automatique, validation par expert, double vérification
 - **Risque moyen :** Avoir non utilisé (obligation de remboursement) → Mitigation : Alertes automatiques 30/60/90 jours
- **Alternatives techniques :**
 - **Alternative A :** Avoir systématique → Simple mais pas toujours approprié
 - **Alternative B :** Avoir optionnel selon le cas → Plus complexe mais fidèle à la réalité

6.3 Portail Client

[] Tâche 2.5 — Portail client (espace client)

- **Description complète :** Créer un portail client accessible via une URL publique où les clients peuvent se connecter (par email et lien sécurisé) pour consulter leurs factures, devis et historique. Fonctionnalités : tableau de bord client avec factures en cours et soldes ; consultation et téléchargement des factures PDF ; formulaire de paiement en ligne (Stripe, PayPal, ou virement) ; messagerie interne avec l'entreprise pour questions sur les factures ; et configuration du profil client (coordonnées, TVA intracommunautaire).

- **Dépendances** : Tâches 0.4, 0.5, 0.6, 1.1, 2.3
 - **Critères d'acceptation** :
 - ☐ Portail client accessible via URL publique
 - ☐ Connexion sécurisée par email + lien magic link
 - ☐ Tableau de bord avec factures et soldes
 - ☐ Téléchargement PDF des factures
 - ☐ Paiement en ligne intégré (Stripe/PayPal)
 - ☐ Messagerie interne client-entreprise
 - ☐ Profil client configurable
 - ☐ Design responsive mobile
 - **Estimation** : 8 jours/hommes
 - **Risques identifiés** :
 - **Risque élevé** : Sécurité du portail client (accès non autorisé) → Mitigation : Tokens magic link expirant, rate limiting, audit de sécurité
 - **Risque moyen** : Paiement en ligne complexe (retours, remboursements) → Mitigation : Webhook Stripe/PayPal, états de paiement synchronisés
 - **Alternatives techniques** :
 - **Alternative A** : Magic link (email) → Simple et sécurisé, mais dépendance email
 - **Alternative B** : Identifiant + mot de passe → Familiar, mais risque de 2FA obligatoire
 - **Alternative C** : SSO via Google/Microsoft → Pratique pour les entreprises, limité pour les particuliers
-

6.4 Module Paiements

[] Tâche 2.6 — Intégration des paiements en ligne

- **Description complète** : Implémenter les paiements en ligne via Stripe (principal) et PayPal (alternative).
Fonctionnalités : paiement fractionné (espèces, CB, virement) ; relance automatique de factures impayées avec escalade (email → SMS → mise en demeure) ; tableau de bord des créances ; prévision des encaissements ; et reconciliation automatique des paiements avec les factures.
- **Dépendances** : Tâches 0.4, 0.6, 1.5, 2.3, 2.5
- **Critères d'acceptation** :
 - ☐ Paiement CB via Stripe fonctionnel
 - ☐ Paiement PayPal fonctionnel
 - ☐ Paiement fractionné géré
 - ☐ Relance automatique avec 3 niveaux d'escalade
 - ☐ Tableau de bord des créances en temps réel
 - ☐ Prévision des encaissements (cash-flow)
 - ☐ Réconciliation automatique paiement/facture

- **Estimation** : 8 jours/hommes
 - **Risques identifiés** :
 - **Risque élevé** : Dépendance à Stripe/PayPal (pannes, blocages) → Mitigation : Multi-processeur, mode dégradé, alertes 24/7
 - **Risque élevé** : Fraude et chargebacks → Mitigation : 3D Secure, scoring Stripe Radar, politique de chargeback
 - **Risque moyen** : Réconciliation échouée → Mitigation : Matching flou, réconciliation manuelle en fallback
 - **Alternatives techniques** :
 - **Alternative A** : Stripe uniquement → Simple mais sans alternative en cas de panne
 - **Alternative B** : Stripe + PayPal → Redondance mais double intégration
 - **Alternative C** : Mangopay → Adapté SEPA mais plus complexe
 - **Alternative D** : Adyen → Entreprise, coûteux pour PME
-

[] Tâche 2.7 — Relance et recouvrement

- **Description complète** : Automatiser le processus de relance des impayés : détection automatique des factures en retard ; envoi de relances progressivement escaladées (rappel poli → relance formelle → mise en demeure) ; génération de lettres de mise en demeure conformes aux exigences légales ; calcul des pénalités de retard et intérêts de commerce ; et reporting du taux de recouvrement.
 - **Dépendances** : Tâches 0.4, 0.6, 2.3, 2.6
 - **Critères d'acceptation** :
 - ☐ Détection automatique des factures en retard (J+1)
 - ☐ 3 niveaux de relance configurables
 - ☐ Mise en demeure conforme au droit français
 - ☐ Calcul automatique des pénalités de retard (loi LME)
 - ☐ Reporting du taux de recouvrement
 - ☐ Arrêt automatique des relances après paiement
 - **Estimation** : 5 jours/hommes
 - **Risques identifiés** :
 - **Risque moyen** : Relance jugée agressive (relations clients) → Mitigation : Ton professionnel, configurabilité, opt-out client
 - **Risque moyen** : Pénalités calculées incorrectement → Mitigation : Validation par expert-comptable, calcul conforme loi LME
 - **Alternatives techniques** :
 - **Alternative A** : Relance automatisée totale → Efficace mais risque relationnel
 - **Alternative B** : Relance semi-automatisée (suggestion + validation) → Plus sûr, légèrement moins efficace
-

Métriques de Progression — Phase 2

Tâche	Statut	Progression
Tâche 2.1 — Création et gestion des devis	☐	0%
Tâche 2.2 — Catalogue produits et services	☐	0%
Tâche 2.3 — Création et gestion des factures	☐	0%
Tâche 2.4 — Gestion des avoirs	☐	0%
Tâche 2.5 — Portail client	☐	0%
Tâche 2.6 — Intégration des paiements	☐	0%
Tâche 2.7 — Relance et recouvrement	☐	0%
Total Phase 2	[]	0%

7. Phase 3 — Module Trésorerie

Progression Globale : [] 0%

Durée estimée : 6-8 semaines

Objectif : Fournir une vision complète de la trésorerie avec un dashboard 360°, des projections de cash-flow, et un système d'alertes proactif.

7.1 Dashboard 360°

[] Tâche 3.1 — Dashboard Trésorerie 360°

- **Description complète :** Développer un dashboard de trésorerie complet offrant une vue à 360° de la situation financière. Inclure : solde global de tous les comptes (banque, caisse, épargne) ; graphique d'évolution du solde sur 12 mois ; revenus et dépenses par catégorie avec comparaison au budget prévisionnel ; graphique de trésorerie prévisionnelle ; top des clients débiteurs et fournisseurs créanciers ; flux de trésorerie entrants et sortants du mois ; et indicateurs clés : BFR (Besoin en Fonds de Roulement), TRS (Trésorerie Réelle Disponible), et ratio de liquidité.
- **Dépendances :** Tâches 0.4, 0.6, 1.5, 1.6, 1.7, 2.3, 2.6
- **Critères d'acceptation :**
 - ☐ Solde global mis à jour en temps réel
 - ☐ Graphique d'évolution 12 mois
 - ☐ Budget prévisionnel configuré et comparé
 - ☐ Trésorerie prévisionnelle visible
 - ☐ Top débiteurs/créanciers avec montant et âge
 - ☐ Indicateurs BFR, TRS, liquidité calculés
 - ☐ Export PDF et Excel du dashboard
 - ☐ Temps de chargement < 3 secondes
- **Estimation :** 7 jours/hommes

- **Risques identifiés :**

- **Risque moyen :** Performance avec gros volumes de transactions → Mitigation : Agrégation serveur, vues matérialisées, cache
- **Risque moyen :** Calcul BFR/TRS incorrect → Mitigation : Validation par expert-comptable, documentation du calcul

- **Alternatives techniques :**

- **Alternative A :** Agrégation en temps réel → Exact mais coûteux en performance
- **Alternative B :** Agrégation nocturne + cache → Plus performant mais données décalées
- **Alternative C :** Matériel SQL (vues matérialisées) → Bon compromis

[] Tâche 3.2 — Gestion budgétaire et prévisionnel

- **Description complète :** Permettre aux utilisateurs de définir des budgets par catégorie, par mois, et par organisation. Le système doit comparer les dépenses réelles avec les budgets et générer des rapports d'écart (sur/d sous-budget). Fonctionnalités : création de budgets annuels ou mensuels ; import de budgets depuis Excel ; alertes d'écart (80%, 100%, 120% de consommation) ; tableaux de bord de suivi budgétaire ; et simulation de scénarios (what-if) pour les dépenses futures.

- **Dépendances :** Tâches 0.4, 0.6, 1.3, 3.1

- **Critères d'acceptation :**

- ☐ Budget création par catégorie et mois
- ☐ Comparaison réel vs prévisionnel automatique
- ☐ Alertes à 80%, 100%, 120% de consommation
- ☐ Import/Export budget Excel
- ☐ Scénarios what-if (projection si dépense x)
- ☐ Historique des budgets modifiés

- **Estimation :** 6 jours/hommes

- **Risques identifiés :**

- **Risque moyen :** Budgets trop rigides → Mitigation : Flexibilité de réallocation, budgets révisables
- **Risque faible :** Comparaison incohérente (base de calcul) → Mitigation : Documentation claire, choix par défaut sensible

- **Alternatives techniques :**

- **Alternative A :** Budget fixe mensuel → Simple mais peu flexible
- **Alternative B :** Budget rolling (continu) → Plus dynamique mais complexe
- **Alternative C :** Budget guidé par IA (Phase 5) → Prognostique mais nécessite données suffisantes

7.2 Cash-flow et Projections

[] Tâche 3.3 — Projection de cash-flow

- **Description complète** : Implémenter un moteur de projection de trésorerie qui anticipe les soldes futurs en se basant sur les factures émises et reçues, les devis probables, les dépenses récurrentes, et les encaissements prévus. Fonctionnalités : projection à 30, 60, 90 jours et 1 an ; scénarios optimiste/pessimiste/réaliste ; import de données saisonnières historiques ; visualisation sous forme de courbe de trésorerie ; et alertes de trésorerie négative prévisionnelle.
 - **Dépendances** : Tâches 0.4, 0.6, 1.7, 2.3, 2.6, 3.1
 - **Critères d'acceptation** :
 - ☐ Projection à 30/60/90 jours et 1 an
 - ☐ 3 scénarios (optimiste, pessimiste, réaliste)
 - ☐ Courbe de trésorerie interactive
 - ☐ Dépenses récurrentes modélisées
 - ☐ Devis probables intégrés
 - ☐ Alerte trésorerie négative prévisionnelle
 - ☐ Historique des projections vs réalité
 - **Estimation** : 8 jours/hommes
 - **Risques identifiés** :
 - **Risque élevé** : Projections erronées → Mitigation : Transparence du modèle, ajustement manuel, historique des projections vs réalité
 - **Risque moyen** : Trop de paramètres → Mitigation : UX progressive (simple → avancé), defaults intelligents
 - **Alternatives techniques** :
 - **Alternative A** : Projection statique (formules simples) → Simple mais imprécise
 - **Alternative B** : Projection ML (Phase 5) → Plus précise mais nécessite données historiques
 - **Alternative C** : Projection hybride (règles + ML) → Bon compromis
-

[] Tâche 3.4 — Gestion des opérations récurrentes

- **Description complète** : Automatiser la gestion des opérations récurrentes : revenus récurrents (abonnements, loyers) ; dépenses récurrentes (loyers, salaires, assurances) ; virements programmés ; et templates d'opérations récurrentes avec possibilité de modifier avant exécution. Le système doit générer automatiquement les transactions correspondantes et notifier l'utilisateur avant exécution.
- **Dépendances** : Tâches 0.4, 0.6, 1.4, 1.7
- **Critères d'acceptation** :
 - ☐ Création d'opérations récurrentes (revenus et dépenses)
 - ☐ Fréquences configurables (hebdo, mensuel, trimestriel, annuel)
 - ☐ Virements programmés avec date et montant
 - ☐ Templates d'opérations
 - ☐ Notification avant exécution (J-3, J-1)
 - ☐ Historique des opérations générées

- ☐ Possibilité de sauter ou modifier avant exécution
 - **Estimation** : 4 jours/hommes
 - **Risques identifiés** :
 - **Risque moyen** : Opération exécutée par erreur → Mitigation : Confirmation préalable, période de grâce, annulation possible
 - **Risque faible** : Oubli d'opération récurrente → Mitigation : Notifications, calendrier visuel
 - **Alternatives techniques** :
 - **Alternative A** : Cron serveur → Fiable mais nécessite surveillance
 - **Alternative B** : File de messages (Bull/Redis Queue) → Scalable mais plus complexe
 - **Alternative C** : Fichier cron système → Simple mais non portable
-

7.3 Alertes Trésorerie

[] Tâche 3.5 — Système d'alertes trésorerie avancé

- **Description complète** : Configurer un système d'alertes intelligentes pour la trésorerie : solde bas seuil configurable ; trésorerie négative prévisionnelle ; gros débit ou crédit inattendu ; facture échéante non réglée ; créance client âgée > 60 jours ; et opportunité d'investissement (excédent de trésorerie). Les alertes doivent être configurables par utilisateur et par organisation.
 - **Dépendances** : Tâches 0.4, 0.6, 1.8, 2.6, 3.1, 3.3
 - **Critères d'acceptation** :
 - ☐ Alerte solde bas (< seuil configurable)
 - ☐ Alerte trésorerie négative prévisionnelle
 - ☐ Alerte gros débit/crédit (> x% du revenu mensuel)
 - ☐ Alerte facture échéante
 - ☐ Alerte créance âgée > 60 jours
 - ☐ Dashboard configurateur d'alertes
 - ☐ Canaux : email, in-app, push (configurable)
 - **Estimation** : 5 jours/hommes
 - **Risques identifiés** :
 - **Risque moyen** : Fatigue d'alertes (trop de notifications) → Mitigation : Pondération, regroupement, configurabilité
 - **Risque faible** : Faux positifs → Mitigation : Seuils calibrés, tests avec données historiques
 - **Alternatives techniques** :
 - **Alternative A** : Seuils statiques → Simple mais inflexible
 - **Alternative B** : Seuils adaptatifs (basés sur historique) → Nécessite Phase 5 (ML)
-

Métriques de Progression — Phase 3

Tâche	Statut	Progression
Tâche 3.1 — Dashboard Trésorerie 360°	☐	0%
Tâche 3.2 — Gestion budgétaire	☐	0%
Tâche 3.3 — Projection cash-flow	☐	0%
Tâche 3.4 — Opérations récurrentes	☐	0%
Tâche 3.5 — Alertes trésorerie avancé	☐	0%
Total Phase 3	[]	0%

8. Phase 4 — Module Conformité

Progression Globale : [] 0%

Durée estimée : 8-10 semaines

Objectif : Assurer la conformité totale avec les normes comptables françaises (PCG, FEC, NF525) et européennes, et mettre en place les outils d'audit.

8.1 Conformité TVA

[] Tâche 4.1 — Gestion TVA et déclarations

- **Description complète :** Implémenter le module de gestion de TVA : calcul automatique de la TVA sur chaque transaction selon le taux applicable (20%, 10%, 5.5%, 2.1%, exonéré) ; gestion de la TVA intracommunautaire (régime simplifié d'abattement) ; TVA reverse charge pour les opérations intra-communautaires ; génération de la déclaration CA3 ; rappels des dates de déclaration ; et export des données pour la déclaration CA3.
- **Dépendances :** Tâches 0.4, 0.6, 1.3, 2.3, 2.4
- **Critères d'acceptation :**
 - ☐ Calcul TVA automatique par taux et type de transaction
 - ☐ TVA intracommunautaire (régime simplifié)
 - ☐ Reverse charge opérations B2B EU
 - ☐ Génération déclaration CA3
 - ☐ Rappels des dates de déclaration (mensuelle/trimestrielle)
 - ☐ Export données CA3 format officiel
 - ☐ Récapitulatif TVA par taux et par période
- **Estimation :** 6 jours/hommes
- **Risques identifiés :**
 - **Risque élevé :** Mauvais taux TVA appliqué → Mitigation : Base de données taux officielle, validation par expert-comptable

- **Risque moyen** : Intracommunautaire mal géré → Mitigation : Vérification N°TVA intracommunautaire, contrôles automatisés

- **Alternatives techniques :**

- **Alternative A** : Calcul manuel par transaction → Simple mais error-prone
- **Alternative B** : Moteur de règles TVA (TaxJar, Avalara) → Externalisé mais coût par transaction
- **Alternative C** : Base de données taux statique → Simple mais nécessite mises à jour manuelles

[] Tâche 4.2 — Gestion de la TVA internationale et e-commerce

- **Description complète** : Pour les entreprises réalisant des ventes en Europe, gérer les règles de TVA applicables : OSS (One Stop Shop) pour les ventes de biens et services distance ; taux de TVA du pays de destination ; génération des déclarations OSS/IOSS ; et conformité aux règles du pays de destination.
- **Dépendances** : Tâches 0.4, 0.6, 4.1
- **Critères d'acceptation** :
 - ☐ OSS (One Stop Shop) implémenté
 - ☐ Taux TVA pays de destination automatique
 - ☐ Génération déclaration OSS mensuelle/trimestrielle
 - ☐ Gestion IOSS pour imports distance < 150€
 - ☐ Validation N°TVA EU avec VIES API
 - ☐ Documentation des règles applicables
- **Estimation** : 5 jours/hommes
- **Risques identifiés** :
 - **Risque élevé** : Non-conformité OSS → Mitigation : Validation par fiscaliste, audit trimestriel
 - **Risque moyen** : API VIES indisponible → Mitigation : Cache des validations, mode dégradé
- **Alternatives techniques** :
 - **Alternative A** : Calcul manuel → Risqué pour du multi-pays
 - **Alternative B** : Service externe (Avalara, TaxJar) → Fiable mais coûteux

8.2 FEC et Bilan

[] Tâche 4.3 — Plan Comptable Général et Grand Livre

- **Description complète** : Implémenter le suivi du Plan Comptable Général (PCG) avec le Grand Livre : saisie des écritures comptables (journal, compte, libellé, montant, pièce justificative) ; solde de chaque compte en temps réel ; grand livre détaillé par compte ; balances (balances âgées, balances nouvelles) ; et journal de l'audit.
- **Dépendances** : Tâches 0.4, 0.6, 1.3, 1.4
- **Critères d'acceptation** :

- ☐ Saisie d'écritures comptables avec journal et compte
 - ☐ Solde en temps réel pour chaque compte PCG
 - ☐ Grand livre détaillé consultable
 - ☐ Balance âgée (comptes actifs et passifs)
 - ☐ Journal de l'audit avec dates et utilisateurs
 - ☐ Import d'écritures depuis FEC
 - **Estimation** : 6 jours/hommes
 - **Risques identifiés** :
 - **Risque élevé** : Écritures en déséquilibre → Mitigation : Validation temps réel débit=crédit, double écriture obligatoire
 - **Risque moyen** : Grand Livre lent avec gros volumes → Mitigation : Pagination, agrégation, indexation
 - **Alternatives techniques** :
 - **Alternative A** : Saisie manuelle → Traditionnel mais fastidieux
 - **Alternative B** : Import FEC → Plus rapide mais dépend de la source
 - **Alternative C** : Synchronisation bancaire automatique → Partiel seulement
-

[] Tâche 4.4 — Génération du Fichier Échanges de Données Comptables (FEC)

- **Description complète** : Générer le fichier FEC (format normalisé par l'administration fiscale française) à partir des données comptables de l'application. Le fichier doit être conforme au schéma officiel (version 2024) et inclure : le fichier d'extrait de comptabilité ; le fichier de l'annexe bilan et compte de résultat ; et les fichiers de TVA. Le système doit valider le fichier FEC selon le schéma XSD avant export.
- **Dépendances** : Tâches 0.4, 0.6, 1.3, 4.3
- **Critères d'acceptation** :
 - ☐ Génération FEC conforme schéma 2024
 - ☐ Validation XSD automatique avant export
 - ☐ Fichiers : Écritures, Annexe Bilan, Annexe Résultat, TVA
 - ☐ Export en format texte séparateur (point-virgule)
 - ☐ Gestion des encodages (UTF-8)
 - ☐ Tests avec validation d'expert-comptable
- **Estimation** : 5 jours/hommes
- **Risques identifiés** :
 - **Risque élevé** : Fichier FEC non conforme rejeté par l'administration → Mitigation : Validation XSD stricte, tests avec données réelles, validation expert
 - **Risque moyen** : Mise à jour du schéma FEC → Mitigation : Versioning du schéma, abstraction pour faciliter les mises à jour
- **Alternatives techniques** :

- **Alternative A** : Génération manuelle depuis Excel → Error-prone, non scalable
- **Alternative B** : Service externe de génération → Dépendance tierce
- **Alternative C** : Bibliothèque open-source FEC → Vérifiable mais nécessite adaptation

[] Tâche 4.5 — Bilan et Compte de Résultat

- **Description complète** : Développer le module de génération automatique du bilan et du compte de résultat à partir du grand livre et des écritures comptables. Inclure : bilan à l'actif (actif immobilisé, actif circulant, trésorerie), passif (capitaux propres, provisions, dettes) ; compte de résultat (chiffre d'affaires, coûts, résultat d'exploitation, résultat financier, résultat exceptionnel) ; comparaison multi-périodes ; et ratios financiers (rentabilité, liquidité, solvabilité).
- **Dépendances** : Tâches 0.4, 0.6, 1.3, 4.3
- **Critères d'acceptation** :
 - ☐ Bilan complet (actif, passif, capitaux propres)
 - ☐ Compte de résultat détaillé
 - ☐ Comparaison N vs N-1, N vs N-2
 - ☐ Ratios financiers calculés
 - ☐ Export PDF et Excel
 - ☐ Validation bilan équilibré (actif = passif + capitaux propres)
 - ☐ Tests avec données de validation fournis par expert-comptable
- **Estimation** : 7 jours/hommes
- **Risques identifiés** :
 - **Risque élevé** : Bilan non équilibré → Mitigation : Contrôle automatique, écritures de régularisation, validation expert
 - **Risque moyen** : Ratios financiers erronés → Mitigation : Documentation des formules, validation croisée
- **Alternatives techniques** :
 - **Alternative A** : Calcul en temps réel → Exact mais coûteux
 - **Alternative B** : Pré-calcul nocturne → Plus performant, données décalées
 - **Alternative C** : Bibliothèque de comptabilité (comptabilidad) → Existe peu en JS/TS

8.3 Audit

[] Tâche 4.6 — Journal d'audit et piste de contrôle

- **Description complète** : Implémenter un journal d'audit complet traçant toutes les modifications de données : qui a fait quoi, quand, depuis où, et quelle était la valeur avant/après modification. Inclure : log systématique des CRUD (Create, Read, Update, Delete) sur toutes les tables sensibles ; filtrabilité par utilisateur, date, action, et table ; conservation des logs pendant 10 ans (obligation légale) ; et export des logs pour l'audit externe.
- **Dépendances** : Tâches 0.4, 0.6, 0.5

- **Critères d'acceptation :**

- ☐ Log automatique de toutes les modifications (CRUD)
- ☐ Informations : utilisateur, timestamp, IP, action, avant/après
- ☐ Filtrage par utilisateur, date, action, table
- ☐ Conservation 10 ans (archivage automatique)
- ☐ Export pour audit externe (CSV, PDF)
- ☐ Logs immuables (tamper-proof)
- ☐ Performance : <5% d'impact sur les opérations principales

- **Estimation :** 6 jours/hommes

- **Risques identifiés :**

- **Risque élevé :** Logs modifiés ou supprimés → Mitigation : Tables d'audit séparées, immuabilité, accès restreint
- **Risque moyen :** Volume de logs excessif → Mitigation : Archivage automatique, compression, rétention intelligente

- **Alternatives techniques :**

- **Alternative A :** Triggers PostgreSQL → Automatique mais difficile à maintenir
- **Alternative B :** Middleware applicatif → Flexible mais dépendance code
- **Alternative C :** Solution externe (Loki, ELK) → Scalable mais infrastructure complexe

[] Tâche 4.7 — Conformité RGPD et Protection des Données

- **Description complète :** Mettre en place les mécanismes de conformité RGPD : consentement explicite des utilisateurs ; droit à l'oubli (suppression complète des données personnelles) ; droit de portabilité (export des données en format structuré) ; registre des traitements de données ; DPO (Délégué à la Protection des Données) désigné ; analyses d'impact (PIA) pour les traitements à risque ; et clauses de confidentialité dans les contrats.

- **Dépendances :** Tâches 0.4, 0.5, 1.1, 4.6

- **Critères d'acceptation :**

- ☐ Consentement explicite enregistré et traçable
- ☐ Droit à l'oubli (suppression 30 jours)
- ☐ Export des données personnelles (JSON, CSV)
- ☐ Registre des traitements de données
- ☐ PIA documentée pour les traitements à risque
- ☐ Politique de confidentialité mise à jour et accessible
- ☐ Notification de fuite de données (<72h)

- **Estimation :** 4 jours/hommes

- **Risques identifiés :**

- **Risque élevé :** Non-conformité RGPD (amendes jusqu'à 4% CA) → Mitigation : Audit juridique, DPO désigné, formation équipe

- **Risque moyen** : Suppression incomplète (données résiduelles) → Mitigation : Mapping complet des données, suppression en cascade

- **Alternatives techniques :**

- **Alternative A** : Outil RGPD externe (OneTrust, TrustArc) → Complet mais coûteux
- **Alternative B** : Implémentation maison → Plus contrôlée mais nécessite expertise juridique

Métriques de Progression — Phase 4

Tâche	Statut	Progression
Tâche 4.1 — TVA et déclarations	☐	0%
Tâche 4.2 — TVA internationale	☐	0%
Tâche 4.3 — Grand Livre PCG	☐	0%
Tâche 4.4 — Génération FEC	☐	0%
Tâche 4.5 — Bilan et Compte de Résultat	☐	0%
Tâche 4.6 — Journal d'audit	☐	0%
Tâche 4.7 — Conformité RGPD	☐	0%
Total Phase 4	[]	0%

9. Phase 5 — Module IA

Progression Globale : [] 0%

Durée estimée : 10-12 semaines

Objectif : Intégrer l'intelligence artificielle pour automatiser la reconnaissance de documents (OCR), la catégorisation intelligente des transactions, et les prédictions financières.

9.1 OCR (Reconnaissance Optique de Caractères)

[] Tâche 5.1 — OCR pour factures et reçus

- **Description complète** : Implémenter un système de reconnaissance optique de caractères (OCR) permettant d'extraire automatiquement les données des factures et reçus. Fonctionnalités : upload d'image (jpg, png, pdf) ; extraction automatique du numéro de facture, date, montant total, TVA, fournisseur ; pré-remplissage des champs de facture ; confirmation et correction manuelle des données extraites ; et apprentissage continu pour améliorer la précision sur les factures récurrentes.
- **Dépendances** : Tâches 0.4, 0.6, 1.4, 2.3, 5.2
- **Critères d'acceptation** :
 - ☐ Upload d'image/PDF de facture
 - ☐ Extraction automatique des champs clés (numéro, date, montant, TVA, fournisseur)
 - ☐ Pré-remplissage des champs de facture

- ☐ Interface de correction des données extraites
 - ☐ Précision > 85% sur factures standard
 - ☐ Apprentissage continu (amélioration sur factures similaires)
 - ☐ Traitement batch (multi-page PDF)
 - ☐ Temps de traitement < 10 secondes par facture
 - **Estimation** : 8 jours/hommes
 - **Risques identifiés** :
 - **Risque élevé** : Précision OCR insuffisante sur factures complexes → Mitigation : Modèle de post-traitement, validation humaine, training data diversifié
 - **Risque moyen** : Coût des appels API OCR → Mitigation : Cache des résultats, batch processing, choix de modèle économique
 - **Risque moyen** : Fausses données acceptées → Mitigation : Validation croisée, seuils de confiance, correction obligatoire si confiance < 90%
 - **Alternatives techniques** :
 - **Alternative A** : API externe (Google Cloud Vision, AWS Textract, Azure Form Recognizer) → Haute précision, coût par page
 - **Alternative B** : Tesseract OCR (open-source) → Gratuit mais moins précis
 - **Alternative C** : Document AI spécialisé (Rossum, Mindee, Nanonets) → Optimisé pour factures, coût variable
 - **Alternative D** : Modèle ML maison (LayoutLM) → Maximum de contrôle, investissement initial élevé
-

[] Tâche 5.2 — Reconnaissance de cartes de visite et documents divers

- **Description complète** : Étendre l'OCR aux cartes de visite, relevés bancaires, et autres documents comptables courants : extraction des coordonnées depuis les cartes de visite ; parsing de relevés bancaires ; classification automatique du type de document ; et stockage des documents originaux avec métadonnées.
- **Dépendances** : Tâches 0.4, 0.6, 5.1
- **Critères d'acceptation** :
 - ☐ Reconnaissance cartes de visite (nom, entreprise, email, téléphone, adresse)
 - ☐ Parsing relevés bancaires (format standard)
 - ☐ Classification automatique du type de document
 - ☐ Stockage document + métadonnées liées
 - ☐ Précision > 80% sur cartes de visite
 - ☐ Recherche par contenu document
- **Estimation** : 4 jours/hommes
- **Risques identifiés** :

- **Risque moyen** : Format de carte de visite varié → Mitigation : ML training, templates, validation manuelle
 - **Risque faible** : Documents multi-langues → Mitigation : OCR multi-langue, détection auto
 - **Alternatives techniques** :
 - **Alternative A** : API Google Cloud Vision → Polyvalent mais coûteux
 - **Alternative B** : Azure Form Recognizer → Excellent pour documents structurés
 - **Alternative C** : Pipeline custom (Tesseract + NLP) → Gratuit mais complexe
-

9.2 Catégorisation ML

[] Tâche 5.3 — Catégorisation automatique des transactions

- **Description complète** : Développer un modèle de Machine Learning pour catégoriser automatiquement les transactions bancaires importées. Le système doit apprendre des habitudes de l'utilisateur : prédiction automatique de la catégorie et du compte ; confiance affichée pour chaque prédiction ; acceptation ou correction en un clic ; feedback (correctif) qui améliore le modèle ; et gestion des cas ambigus avec règle de fallback (catégorie la plus probable + suggestion).
- **Dépendances** : Tâches 0.4, 0.6, 1.3, 1.5, 5.1
- **Critères d'acceptation** :
 - ☐ Prédiction automatique de la catégorie pour chaque transaction importée
 - ☐ Score de confiance affiché (>80% auto-applicable)
 - ☐ Correction en un clic avec feedback au modèle
 - ☐ Amélioration progressive de la précision
 - ☐ Précision > 90% après 3 mois d'utilisation
 - ☐ Règle de fallback configurable
 - ☐ Support multi-comptes et multi-organisations
 - ☐ Dashboard de précision du modèle
- **Estimation** : 8 jours/hommes
- **Risques identifiés** :
 - **Risque élevé** : Mauvaise précision initiale → Mitigation : Phase de warm-up, règles basées sur les mots-clés, onboarding guidé
 - **Risque moyen** : Catégorisation biaisée → Mitigation : Audit régulier, diversité des données d'entraînement
 - **Risque moyen** : Cold start (nouveau utilisateur) → Mitigation : Règles basées sur les mots-clés + suggestions manuelles initiales
- **Alternatives techniques** :
 - **Alternative A** : Règles basées sur les mots-clés → Simple et prévisible, non adaptatif
 - **Alternative B** : ML supervisé (Naive Bayes, Random Forest) → Bon compromis, entraînable
 - **Alternative C** : Deep Learning (Transformers) → Plus précis mais coûteux et complexe

- **Alternative D** : Solution externe (Codat, Soldo) → Externalisé mais coûteux et moins personnalisable
-

[] Tâche 5.4 — Détection d'anomalies et transactions suspectes

- **Description complète** : Implémenter un système de détection d'anomalies utilisant le Machine Learning pour identifier les transactions inhabituelles : montant anormalement élevé par rapport à l'historique ; transaction à un fournisseur jamais utilisé ; fréquence inhabituelle de transactions ; et transactions en dehors des heures ouvrables normales. Le système doit alerter l'utilisateur et proposer une vérification.
 - **Dépendances** : Tâches 0.4, 0.6, 1.5, 5.3
 - **Critères d'acceptation** :
 - ☐ Détection des transactions anormalement élevées
 - ☐ Détection fournisseurs inhabituels
 - ☐ Détection fréquence anormale
 - ☐ Alertes in-app et email
 - ☐ Score de risque par transaction
 - ☐ Seuil de sensibilité configurable
 - ☐ Historique des anomalies détectées et traitées
 - ☐ <5% faux positifs après calibration
 - **Estimation** : 5 jours/hommes
 - **Risques identifiés** :
 - **Risque moyen** : Trop de faux positifs → Mitigation : Calibration progressive, user feedback
 - **Risque moyen** : Menace de jour 0 non détectée → Mitigation : Multi-modèles, règles complémentaires
 - **Alternatives techniques** :
 - **Alternative A** : Seuils statiques → Simple mais peu adaptatif
 - **Alternative B** : Isolation Forest (ML) → Efficace pour détection d'anomalies
 - **Alternative C** : Autoencoder → Plus sophistiqué mais nécessite plus de données
-

9.3 Prédictions Financières

[] Tâche 5.5 — Prédictions de revenus et dépenses futures

- **Description complète** : Créer un moteur de prédiction basé sur les données historiques pour anticiper les revenus et dépenses futurs. Fonctionnalités : prédiction des revenus réguliers (abonnements, contrats) ; prédiction des dépenses saisonnières ; prédiction du cash-flow mensuel ; suggestions de placement de l'excédent de trésorerie ; et rapport de précision des prédictions passées vs réalité.
- **Dépendances** : Tâches 0.4, 0.6, 1.5, 3.1, 3.3, 5.3
- **Critères d'acceptation** :

- ☐ Prédiction revenus 1-6 mois avec intervalle de confiance
 - ☐ Prédiction dépenses saisonnières
 - ☐ Prédiction cash-flow mensuel
 - ☐ Suggestions de placement trésorerie
 - ☐ Rapport de précision historique
 - ☐ Interface de visualisation interactive
 - ☐ Ajustement manuel des prédictions possible
 - ☐ Précision > 75% après 6 mois de données
 - **Estimation** : 7 jours/hommes
 - **Risques identifiés** :
 - **Risque élevé** : Prédictions erronées (mauvaise foi perçue) → Mitigation : Intervalle de confiance, transparence du modèle, avertissements explicites
 - **Risque moyen** : Données insuffisantes → Mitigation : Minimum 3 mois de données requises, règles de fallback
 - **Alternatives techniques** :
 - **Alternative A** : Moyennes mobiles → Simple mais imprécis pour variations
 - **Alternative B** : Prophet (Meta) → Gère saisonnalité et tendances, open-source
 - **Alternative C** : ARIMA/SARIMA → Classique mais complexe à configurer
 - **Alternative D** : LSTM/Transformers → Puissant mais gourmand en données
-

[] Tâche 5.6 — Conseils financiers automatisés (Insights IA)

- **Description complète** : Fournir des recommandations financières personnalisées basées sur les données de l'utilisateur : optimisation du BFR ; suggestion de négociation de délais fournisseurs ; identification de souscriptions inutilisées ; recommandation de structure fiscale ; et alertes de trésorerie proactive. Les insights doivent être présentés comme des cartes dans le dashboard avec des explications claires et actionnables.
- **Dépendances** : Tâches 0.4, 0.6, 3.1, 3.3, 5.3, 5.5
- **Critères d'acceptation** :
 - ☐ Insights BFR optimisé
 - ☐ Suggestions négociation délais fournisseurs
 - ☐ Identification souscriptions inutilisées
 - ☐ Recommandations structure fiscale
 - ☐ Alertes proactives trésorerie
 - ☐ Présentation sous forme de cartes explicatives
 - ☐ Score de pertinence des conseils
 - ☐ Feedback utilisateur (utile/non utile)
- **Estimation** : 6 jours/hommes
- **Risques identifiés** :

- **Risque élevé** : Conseils financiers erronés (responsabilité légale) → Mitigation : Disclaimer obligatoire, validation expert, non-recommandation personnalisée type conseil en investissement
- **Risque moyen** : Insights non pertinents → Mitigation : User feedback, amélioration continue, focus sur les domaines à fort impact

• **Alternatives techniques :**

- **Alternative A** : Règles métier statiques → Simple et prévisible
- **Alternative B** : LLM (GPT, Claude) pour génération d'insights → Naturel mais hallucination possible
- **Alternative C** : ML supervisé sur historique → Plus fiable mais nécessite données labélisées

Métriques de Progression — Phase 5

Tâche	Statut	Progression
Tâche 5.1 — OCR factures et reçus	☐	0%
Tâche 5.2 — Reconnaissance cartes de visite	☐	0%
Tâche 5.3 — Catégorisation ML	☐	0%
Tâche 5.4 — Détection anomalies	☐	0%
Tâche 5.5 — Prédictions financières	☐	0%
Tâche 5.6 — Conseils financiers IA	☐	0%
Total Phase 5	[]	0%

10. Phase 6 — Infrastructure & Ops

Progression Globale : [] 0%

Durée estimée : 6-8 semaines

Objectif : Containeriser l'application, finaliser le pipeline CI/CD, mettre en place les tests complets, et déployer la solution en production avec un monitoring robuste.

10.1 Dockerisation et Orchestration

[] **Tâche 6.1 — Dockerisation complète**

- **Description complète** : Créer des Dockerfiles et docker-compose.yml pour tous les services de l'application : backend API, frontend SPA, base de données PostgreSQL, Redis, et services auxiliaires. Chaque service doit avoir son propre Dockerfile optimisé (multi-stage build pour réduire la taille d'image). Configurer les volumes persistants, les réseaux Docker, et les variables d'environnement par service. Définir des images de base sécurisées et minimisées (Alpine, distroless).
- **Dépendances** : Tâches 0.1, 0.2, 0.3, 0.6
- **Critères d'acceptation** :
 - ☐ Dockerfile backend (multi-stage, production-ready)

- ☐ Dockerfile frontend (build + serve)
 - ☐ Docker-compose.yml complet (5+ services)
 - ☐ Volumes persistants configurés
 - ☐ Réseau Docker inter-services fonctionnel
 - ☐ Images < 500Mo chacune (backend, frontend)
 - ☐ Build local en 1 commande (**docker compose up**)
 - ☐ Health checks configurés par service
 - ☐ Tests de démarrage depuis zéro (fresh clone) réussis
 - **Estimation** : 5 jours/hommes
 - **Risques identifiés** :
 - **Risque moyen** : Images Docker volumineuses → Mitigation : Multi-stage builds, Alpine, .dockerignore
 - **Risque moyen** : Problèmes de permissions → Mitigation : Utilisateur non-root dans Docker, configuration des volumes
 - **Risque faible** : Build lente → Mitigation : Cache des couches, build parallèle
 - **Alternatives techniques** :
 - **Alternative A** : Docker standard → Universel, bien supporté
 - **Alternative B** : Podman → Rootless, plus sécurisé
 - **Alternative C** : NixOS/Nix → Reproductible mais adoption faible
 - **Alternative D** : VM (Vagrant) → Plus lourd mais isolation totale
-

[] Tâche 6.2 — Kubernetes et orchestration de production

- **Description complète** : Configurer un cluster Kubernetes (ou équivalent comme Docker Swarm pour les petites échelles) pour le déploiement en production : manifests de déploiement, services, configmaps, secrets, persistent volume claims, horizontal pod autoscaling, et ingress controller (Nginx Ingress ou Traefik). Configurer les stratégies de rollout (rolling update) et rollback.
- **Dépendances** : Tâches 0.1, 0.2, 0.3, 0.6, 6.1
- **Critères d'acceptation** :
 - ☐ Manifests K8s complets (deploy, service, ingress, configmap, secret)
 - ☐ Horizontal Pod Autoscaler configuré
 - ☐ Persistent Volume pour base de données
 - ☐ Ingress controller fonctionnel (HTTPS automatique)
 - ☐ Rolling updates configurées
 - ☐ Rollback fonctionnel en cas de déploiement échoué
 - ☐ Ressources limitées par pod (requests/limits)
 - ☐ PodDisruptionBudget pour haute disponibilité
- **Estimation** : 6 jours/hommes
- **Risques identifiés** :

- **Risque élevé** : Complexité K8s → Mitigation : Managed K8s (EKS, GKE, AKS), ou Docker Swarm pour MVP
 - **Risque moyen** : Configuration de ressources inadaptée → Mitigation : Tests de charge, monitoring, ajustement itératif
 - **Alternatives techniques** :
 - **Alternative A** : Kubernetes managé (EKS/GKE/AKS) → Robuste mais coût élevé
 - **Alternative B** : Docker Swarm → Simple mais limité en fonctionnalités
 - **Alternative C** : Docker Compose + reverse proxy → Pour petites charges uniquement
 - **Alternative D** : Serverless (AWS Lambda, Vercel, Cloudflare Workers) → Scalable mais cold starts, limitations
-

10.2 Tests et Qualité

[] Tâche 6.3 — Suite de tests complète

- **Description complète** : Mettre en place une stratégie de tests complète couvrant tous les niveaux : tests unitaires (Jest, Vitest, PyTest) ; tests d'intégration (supertest, pytest) ; tests end-to-end (Cypress, Playwright) ; tests de performance (k6, Artillery) ; et tests de sécurité (OWASP ZAP, Snyk). Définir les seuils de couverture minimale (80% unitaire, 60% intégration) et l'intégration dans le pipeline CI/CD.
- **Dépendances** : Tâches 0.3, 0.6, 6.1
- **Critères d'acceptation** :
 - ☐ Tests unitaires avec couverture > 80%
 - ☐ Tests d'intégration > 50 scénarios
 - ☐ Tests E2E (Cypress/Playwright) > 20 scénarios critiques
 - ☐ Tests de performance de base (k6/Artillery)
 - ☐ Tests de sécurité (OWASP ZAP scan)
 - ☐ Intégration dans le pipeline CI/CD
 - ☐ Seuils de qualité bloquants configurés (couverture, E2E)
 - ☐ Documentation des tests dans docs/testing.md
- **Estimation** : 8 jours/hommes
- **Risques identifiés** :
 - **Risque moyen** : Tests E2E lents/fragiles → Mitigation : Tests intelligents, retry, isolation des données
 - **Risque moyen** : Couverture insuffisante → Mitigation : Coverage report bloquant sous le seuil
 - **Risque faible** : Tests de performance non représentatifs → Mitigation : Profils de charge réalistes
- **Alternatives techniques** :
 - **Alternative A** : Cypress (E2E) → Excellent UX mais JavaScript uniquement
 - **Alternative B** : Playwright → Multi-langage, plus récent
 - **Alternative C** : Selenium → Mature mais lent
 - **Alternative D** : k6 (performance) → Moderne, JavaScript, cloud optionnel

[] Tâche 6.4 — Linting, formatting et analyse statique

- **Description complète** : Configurer les outils de qualité de code : ESLint + Prettier (frontend) ; Ruff/Black/PyLint (backend si Python) ; ou SonarLint (multi-langage) ; analyse statique de sécurité (Semgrep, SonarQube) ; et PR checks automatiques. Définir des règles strictes et les intégrer dans les éditeurs des développeurs via configuration partagée.
 - **Dépendances** : Tâches 0.1, 0.3
 - **Critères d'acceptation** :
 - ☐ ESLint/Prettier configurés et partagés
 - ☐ Analyse statique SonarQube/SonarCloud
 - ☐ PR checks bloquants sur erreurs critiques
 - ☐ Configuration partagée dans `/.lintstagedrc` et `/.editorconfig`
 - ☐ Pre-commit hooks (lint-staged, husky)
 - ☐ Zéro erreur critique dans l'analyse statique initiale
 - ☐ Documentation des règles dans `docs/coding-standards.md`
 - **Estimation** : 3 jours/hommes
 - **Risques identifiés** :
 - **Risque faible** : Résistance de l'équipe aux règles strictes → Mitigation : Gradual adoption, fix d'erreurs existantes progressif
 - **Risque faible** : Pre-commit hooks lents → Mitigation : Concurrent execution, selective hooks
 - **Alternatives techniques** :
 - **Alternative A** : ESLint + Prettier → Standard JavaScript/TypeScript
 - **Alternative B** : Biome → Alternatif plus rapide, Rust-based
 - **Alternative C** : SonarQube → Complet mais nécessite serveur dédié
 - **Alternative D** : CodeClimate → SaaS, simple mais coût récurrent
-

10.3 Monitoring et Observabilité

[] Tâche 6.5 — Monitoring et alerting en production

- **Description complète** : Mettre en place un système de monitoring complet : métriques applicatives (temps de réponse, taux d'erreur, throughput) ; métriques infrastructure (CPU, RAM, disque, réseau) ; monitoring de la base de données (requêtes lentes, connexions, taille) ; monitoring des services tiers (Stripe, API bancaires) ; et alerting intelligent (PagerDuty, OpsGenie) avec escalation et routage.
- **Dépendances** : Tâches 0.2, 0.3, 0.6, 6.1, 6.2
- **Critères d'acceptation** :
 - ☐ Métriques applicatives en temps réel (Prometheus/Grafana)
 - ☐ Métriques infrastructure (CPU, RAM, disque)
 - ☐ Monitoring DB (requêtes lentes, connexions)

- ☐ Monitoring services tiers (SLA)
 - ☐ Alertes configurées avec escalation
 - ☐ Dashboard Grafana fonctionnel
 - ☐ Uptime monitoring (UptimeRobot, Pingdom)
 - ☐ Alertes 24/7 avec PagerDuty/OpsGenie
 - ☐ Temps de détection d'incident < 5 minutes
- **Estimation** : 6 jours/hommes
 - **Risques identifiés** :
 - **Risque élevé** : Alertes fatiguées (alert fatigue) → Mitigation : Seuils intelligents, routage par sévérité, réduction du bruit
 - **Risque moyen** : Données de monitoring incomplètes → Mitigation : Métriques personnalisées, dashboards itératifs
 - **Alternatives techniques** :
 - **Alternative A** : Prometheus + Grafana → Open-source, flexible, standard
 - **Alternative B** : Datadog → Tout-en-un mais coûteux
 - **Alternative C** : New Relic → Bon APM mais cher
 - **Alternative D** : CloudWatch (AWS) → Intégré mais limité multi-cloud
-

[] Tâche 6.6 — Logging centralisé et traçabilité

- **Description complète** : Configurer un système de logging centralisé : collecte des logs de tous les services (Fluentd, Fluent Bit, Filebeat) ; stockage et indexation (Elasticsearch/OpenSearch) ; visualisation et recherche (Kibana/Grafana Loki) ; et retention policy (30 jours hot, 1 an warm, 7 ans cold/archive). Garantir la traçabilité des actions utilisateurs pour l'audit et la conformité.
- **Dépendances** : Tâches 0.2, 0.3, 0.6, 6.1, 6.5
- **Critères d'acceptation** :
 - ☐ Logs collectés de tous les services
 - ☐ Recherche et filtrage par service, niveau, date, utilisateur
 - ☐ Dashboard de logs opérationnels
 - ☐ Retention 30 jours hot, 1 an warm
 - ☐ Archive 7 ans conforme réglementation
 - ☐ Logs structurés (JSON)
 - ☐ Correlation ID pour chaque requête API
 - ☐ Alertes basées sur les logs (erreurs critiques)
- **Estimation** : 5 jours/hommes
- **Risques identifiés** :
 - **Risque moyen** : Volume de logs excessif → Mitigation : Filtrage niveau INFO+, sampling DEBUG, compression

- **Risque moyen** : Données sensibles dans les logs → Mitigation : Masking automatique (PII), politique de log

- **Alternatives techniques :**

- **Alternative A** : ELK Stack (Elasticsearch + Logstash + Kibana) → Complet mais lourd
 - **Alternative B** : Loki + Grafana → Léger, cloud-native
 - **Alternative C** : CloudWatch Logs → Intégré AWS, simple
 - **Alternative D** : Papertrail → SaaS simple mais limité
-

[] Tâche 6.7 — Backup et reprise après sinistre

- **Description complète** : Mettre en place une stratégie de sauvegarde et de reprise après sinistre : backups automatiques quotidiens de la base de données (rétention 30 jours) ; backups des fichiers (S3 versioning) ; backup des configurations et secrets ; plan de reprise après sinistre (RPO < 1h, RTO < 4h) ; tests de restauration trimestriels ; et documentation de procédures de recovery.
 - **Dépendances** : Tâches 0.2, 0.3, 0.4, 6.1, 6.5
 - **Critères d'acceptation** :
 - ☐ Backup DB automatique quotidien (30 jours rétention)
 - ☐ Backup fichiers S3 versioning activé
 - ☐ Backup configurations et secrets chiffrés
 - ☐ Plan DR documenté (RPO < 1h, RTO < 4h)
 - ☐ Test de restauration réussi (trimestriel)
 - ☐ Procédures de recovery documentées
 - ☐ Backup monitoring (alerte si backup échoué)
 - ☐ Cross-region backup pour disaster recovery
 - **Estimation** : 4 jours/hommes
 - **Risques identifiés** :
 - **Risque élevé** : Perte de données (backup échoue silencieusement) → Mitigation : Monitoring des backups, alertes, tests de restauration
 - **Risque moyen** : RTO trop long → Mitigation : Infrastructure as Code, automation du recovery
 - **Alternatives techniques** :
 - **Alternative A** : Backup pg_dump automatique → Simple et fiable pour PostgreSQL
 - **Alternative B** : Backup WAL-G/ Wal-E → Continue, plus complexe
 - **Alternative C** : Backup snapshots cloud (AWS RDS) → Managé mais dépendance cloud
 - **Alternative D** : Réplication multi-master → Haute disponibilité mais complexe
-

[] Tâche 6.8 — Documentation technique et opérationnelle

- **Description complète** : Créer une documentation technique et opérationnelle complète : architecture technique (diagrammes C4) ; API documentation (Swagger/OpenAPI) ; procédures opérationnelles (onboarding, déploiement, rollback, incident) ; runbooks pour les incidents courants ; FAQ technique ; et

documentation de déploiement. La documentation doit être versionnée avec le code et accessible à tous les membres de l'équipe.

- **Dépendances** : Tâches 0.1, 0.2, 0.3, 0.6, 6.1, 6.5, 6.6
- **Critères d'acceptation** :
 - ☐ Architecture documentée (diagrammes C4 niveau 1-3)
 - ☐ API documentation (Swagger/OpenAPI) auto-générée
 - ☐ Procédures opérationnelles documentées
 - ☐ 10+ runbooks d'incidents
 - ☐ Guide d'onboarding des développeurs
 - ☐ Documentation déployée (MkDocs/Docusaurus)
 - ☐ Documentation versionnée avec le code
 - ☐ Dernière mise à jour < 30 jours
- **Estimation** : 4 jours/hommes
- **Risques identifiés** :
 - **Risque moyen** : Documentation non maintenue → Mitigation : Linter de docs (markdownlint), review dans PR
 - **Risque faible** : Documentation incompréhensible → Mitigation : Review par les pairs, exemples concrets
- **Alternatives techniques** :
 - **Alternative A** : MkDocs → Simple, Python-based, Material theme
 - **Alternative B** : Docusaurus → React-based, intégré au code
 - **Alternative C** : Confluence → Enterprise mais externalisé
 - **Alternative D** : Notion → Flexible mais non versionné

Métriques de Progression — Phase 6

Tâche	Statut	Progression
Tâche 6.1 — Dockerisation	☐	0%
Tâche 6.2 — Kubernetes	☐	0%
Tâche 6.3 — Suite de tests	☐	0%
Tâche 6.4 — Linting et analyse statique	☐	0%
Tâche 6.5 — Monitoring et alerting	☐	0%
Tâche 6.6 — Logging centralisé	☐	0%
Tâche 6.7 — Backup et reprise après sinistre	☐	0%
Tâche 6.8 — Documentation	☐	0%
Total Phase 6	[]	0%

11. Plan de Mitigation des Risques

11.1 Risques Stratégiques

#	Risque	Impact	Probabilité	Mitigation	Plan de Contingence
R1	Perte de financement / budget réduit	Critique	Faible	Planification par sprints livrables, MVP first	Réduire le scope aux phases 0-1 uniquement
R2	Turnover développeur clé	Élevé	Moyenne	Documentation exhaustive, pairing programming, documentation des connaissances	Recrutement externe, externalisation temporaire
R3	Changement réglementaire majeur	Élevé	Moyenne	Veille juridique continue, architecture modulaire	Adaptation rapide des modules impactés
R4	Décision erronée d'architecture	Critique	Faible	POC avant décision, revue d'architecture externe	Refactoring incrémental, abstractions

11.2 Risques Techniques

#	Risque	Impact	Probabilité	Mitigation	Plan de Contingence
R5	Vulnérabilité de sécurité	Critique	Moyenne	Audit de sécurité (Phase 0), OWASP Top 10, pentests réguliers	Incident response plan, isolation des services
R6	Fuite de données multi-tenancy	Critique	Faible	RLS PostgreSQL, tests d'isolation systématiques	Migration de schéma d'urgence, notification RGPD
R7	Performance insuffisante à échelle	Élevé	Moyenne	Architecture scalable dès le départ, load testing régulier	Optimisation des requêtes, ajout de cache, CDN
R8	Dépendance API tierce (Stripe, Plaid)	Élevé	Moyenne	Abstraction layer, multi-fournisseur, mode dégradé	Switch manuel vers un autre fournisseur
R9	Indisponibilité infrastructure cloud	Élevé	Faible	Multi-region, failover automatique	Plan de disaster recovery, RTO < 4h

#	Risque	Impact	Probabilité	Mitigation	Plan de Contingence
R10	Échec OCR/IA (précision insuffisante)	Élevé	Moyenne	Seuil de confiance, correction manuelle obligatoire, fallback règles	Revenir à la saisie manuelle temporairement
R11	Incohérence des données comptables	Critique	Faible	Double écriture obligatoire, reconciliation continue, audit trail	Correction manuelle avec validation expert

11.3 Risques Opérationnels

#	Risque	Impact	Probabilité	Mitigation	Plan de Contingence
R12	Retard de livraison	Élevé	Moyenne	Sprints agiles, scrums quotidiens, burndown charts	Réduction de scope, dépriorisation des fonctionnalités non critiques
R13	Non-conformité réglementaire	Critique	Faible	Audit juridique externe, DPO nommé, veille réglementaire	Consultation urgente expert-comptable, gel des fonctionnalités
R14	Mauvaise adoption utilisateur	Élevé	Moyenne	User research continue, UX testing, onboarding guidé	Simplification de l'UX, formation dédiée
R15	Dépassement budgétaire	Élevé	Moyenne	Suivi hebdomadaire des coûts, buffer de 20%	Réduction des fonctionnalités non essentielles

11.4 Matrice de Risques Globale

Impact élevé	R4, R5, R8, R12, R14 R12, R14
Impact moyen	R2, R3, R7, R10, R13, R15 R2, R3, R7, R10, R13
Impact faible	R1 R1
Probabilité élevée	- R12, R14, R15
Probabilité moyenne	R2, R5, R7, R8, R10, R12, R13, R14, R15 R5, R7, R8, R10, R13
Probabilité faible	R4, R5, R6, R9, R11, R13 R6, R9, R11

12. Glossaire

Terme	Définition
SaaS	Software as a Service — logiciel hébergé et fourni via Internet sous forme d'abonnement

Terme	Définition
FEC	Fichier Échanges de Données Comptables — format normalisé par l'administration fiscale française
NF525	Norme française spécifiant les conditions de délivrance des tickets fiscaux et factures
BFR	Besoin en Fonds de Roulement — différence entre les stocks/créances et les dettes fournisseurs
TRS	Trésorerie Réelle Disponible — cash + équivalents cash + actifs à court terme - dettes à court terme
PCG	Plan Comptable Général — référentiel comptable français standard
PSD2	Payment Services Directive 2 — directive européenne sur les services de paiement et Open Banking
OSS	One Stop Shop — système de TVA simplifié pour les ventes distance intra-UE
RGPD	Règlement Général sur la Protection des Données — réglementation européenne sur la protection des données personnelles
RPO	Recovery Point Objective — quantité maximale de données pouvant être perdues
RTO	Recovery Time Objective — temps maximum pour restaurer un système après une panne
JWT	JSON Web Token — standard de transmission sécurisée d'informations entre deux entités
K8S	Kubernetes — système d'orchestration de conteneurs
OCR	Optical Character Recognition — reconnaissance optique de caractères
ML	Machine Learning — apprentissage automatique
API	Application Programming Interface — interface de programmation applicative
CRUD	Create, Read, Update, Delete — quatre opérations de base sur les données
SLA	Service Level Agreement — accord de niveau de service
CI/CD	Continuous Integration / Continuous Deployment — intégration et déploiement continus
DDOS	Distributed Denial of Service — attaque par déni de service distribué
CORS	Cross-Origin Resource Sharing — mécanisme de partage de ressources cross-origin
XSS	Cross-Site Scripting — attaque par injection de scripts
CSRF	Cross-Site Request Forgery — attaque par falsification de requête
OWASP	Open Web Application Security Project — organisation de sécurité applicative web

Note finale : Cette roadmap est un document vivant qui doit être revue et mise à jour à chaque sprint de rétrospective. Les pourcentages de progression doivent être mis à jour hebdomadairement par le Scrum Master ou le chef de projet. Les risques doivent être réévalués mensuellement en comité de pilotage.